

Build a Full Stack Project in Django

Comprehensive Study Notes & Architecture Guide

Source Reference: Chai aur Code (YouTube Tutorial) | **Topic:** Full-Stack Beginner Django Project (Twitter/X Clone)

1. Project Architecture & Setup

Building a robust Django project requires proper environment setup, dependency management, and configuration for handling static assets and media uploads.

Virtual Environment & Installation

```
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
pip install django Pillow
```

Project Initialization & Settings Configuration

Initialize the project using `django-admin startproject chaihky`. In `settings.py`, configure directories for serving media and static files:

```
MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')

STATIC_URL = 'static/'
STATICFILES_DIRS = [os.path.join(BASE_DIR, 'static')]
```

Main URL Routing

Include media serving routing in the primary `urls.py` configuration file:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('tweet/', include('tweet.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

2. App Creation & Database Models

Create a dedicated app for handling tweets using `python manage.py startapp tweet`, and register it inside `INSTALLED_APPS` in `settings.py`.

Model Definition (models.py)

Define the database schema connecting each tweet to Django's built-in `User` model via a `ForeignKey` constraint.

```
from django.db import models
from django.contrib.auth.models import User

class Tweet(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    text = models.TextField(max_length=240)
    photo = models.ImageField(upload_to='photos/', blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.user.username} - {self.text[:10]}"
```

Migrations & Admin Registration

```
python manage.py makemigrations tweet
python manage.py migrate
python manage.py createsuperuser
```

Register the model in `admin.py` to manage records through the Django administrative dashboard.

3. Forms Integration

Leverage Django's built-in `ModelForm` capability inside `forms.py` to automatically construct inputs for model fields.

```

from django import forms
from .models import Tweet
from django.contrib.auth.models import User
from django.contrib.auth.forms import UserCreationForm

class TweetForm(forms.ModelForm):
    class Meta:
        model = Tweet
        fields = ['text', 'photo']

class UserRegistrationForm(UserCreationForm):
    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

```

4. Views & CRUD Operations

Implement core business logic inside `views.py` covering listing, creation, editing, deletion, and user registration.

Key View Functions

- **tweet_list:** Fetches all tweets ordered by creation timestamp and passes them to the template.
- **tweet_create:** Uses `@login_required` decorator. Handles POST requests with multi-part file data, assigns the active user, and saves.
- **tweet_edit:** Utilizes `get_object_or_404` ensuring users can only modify their own tweets using form instances.
- **tweet_delete:** Removes specific tweet records upon explicit user confirmation.
- **register:** Manages user account creation and automatically logs in the newly registered user.

5. Templates & UI Layout

Build a clean user interface utilizing Bootstrap for styling, template inheritance for modular layouts, and Jinja tags for dynamic data rendering.

- **layout.html:** Master template featuring a navigation bar, dark theme support, and content blocks.
- **tweet_list.html:** Iterates through tweets using conditional checks to display edit/delete buttons exclusively to post authors.
- **tweet_form.html:** Dynamically handles both create and edit operations, incorporating `enctype="multipart/form-data"` and `{% csrf_token %}` for security.